Reinforcement Learning in a Warehouse

Theory in brief, and a measured comparison of
Q-learning, SARSA and REINFORCE

Mariam Bouzidi

Abstract

Reinforcement learning (RL) papers often report a single training run judged by eye. This note takes a different route: it picks a navigation problem small enough to be *solved exactly*, and uses the exact solution to grade three learning algorithms over many random seeds. After a compact summary of the theory (MDPs, Bellman equations, temporal-difference learning, policy gradients), it reports what the measurements show: Q-learning recovers shortest paths from 99.6% of start squares and is robust to its hyper-parameters; SARSA is far more fragile, for a reason the exact solution makes explicit (exploring is dangerous in this environment); the policy converges long before the value function; and REINFORCE works but needs roughly five times more episodes, with a baseline clearly reducing variance. All code, figures and numbers are reproducible from the accompanying repository.

1 Theory in brief

1.1 MDPs and optimality

An agent interacts with an environment modelled as a Markov decision process $\langle S, A, P, R, \gamma\rangle$: states S, actions A, transition probabilities $P(s' \mid s, a)$, rewards $R(s, a, s')$ and a discount $\gamma \in [0, 1)$. The Markov property says the next state depends only on the current state and action. The goal is a policy π maximising the expected discounted return

$$G_t = \sum_{k=0}^{\infty} \gamma^k \, r_{t+k+1}.$$

The action-value function $Q^\pi(s, a) = \mathbb{E}_\pi[G_t \mid s_t = s, a_t = a]$ of an optimal policy π^*, written Q^*, satisfies the Bellman optimality equation

$$Q^*(s, a) = \sum_{s'} P(s' \mid s, a)\Big[R(s, a, s') + \gamma \max_{a'} Q^*(s', a')\Big], \tag{1}$$

and $\pi^*(s) \in \arg\max_a Q^*(s, a)$. When the model is known and finite, *value iteration* (repeatedly applying the right-hand side of (1)) converges to Q^* because the Bellman operator is a γ-contraction.

1.2 Learning without the model

When P and R are unknown, Q is learned from sampled transitions (s, a, r, s') by temporal-difference updates, $Q(s, a) \leftarrow Q(s, a) + \alpha\,[y - Q(s, a)]$, with an ε-greedy behaviour policy (a random action with probability ε). The two algorithms differ only in the target y:

	target y	what it converges to
Q-learning (off-policy)	$r + \gamma \max_{a'} Q(s', a')$	Q^* (tabular, Robbins–Monro steps)
SARSA (on-policy)	$r + \gamma\, Q(s', a'),\ a' \sim \varepsilon$-greedy	Q^*_ε for fixed ε; Q^* if $\varepsilon \to 0$ (GLIE)

The bootstrap term is zero when s' is terminal. The distinction that matters below is that SARSA evaluates the policy it *follows*, exploration included. Its fixed point is therefore the best action-value function achievable *while still exploring*:

$$Q^*_\varepsilon(s,a) = \sum_{s'} P(s' \mid s,a)\Big[R + \gamma\Big((1-\varepsilon)\max_{a'} Q^*_\varepsilon(s',a') + \varepsilon\,\tfrac{1}{|A|}\sum_{a'} Q^*_\varepsilon(s',a')\Big)\Big].$$

This too can be computed exactly on a small problem.

1.3 Policy gradients

Instead of values, one can parametrise the policy, $\pi_\theta(a \mid s)$, and ascend $J(\theta) = \mathbb{E}_{\pi_\theta}[G_0]$. The log-derivative trick gives

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta}\Big[\sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)\,(G_t - b(s_t))\Big],$$

valid for any baseline $b(s)$ that does not depend on the action. The baseline leaves the gradient unbiased but reduces its variance. REINFORCE (Williams, 1992) performs stochastic ascent with this estimate; for a tabular softmax policy, $\nabla_{\theta_{s,\cdot}} \log \pi_\theta(a \mid s) = \mathbf{1}_a - \pi_\theta(\cdot \mid s)$.

2 Demo: an exactly solvable warehouse

Everything in this section is implemented in `rl_warehouse.py` and run by the accompanying notebook; every number is produced by seeded code and stored in `results.json`.

2.1 Setup

A robot must reach the packaging area of an 11×11 warehouse (Figure 1). The state is the grid square (121 states) and there are four actions. The reward received on entering a square is $+100$ at the goal, -1 on an aisle square (so shorter is better), and -100 on a shelf, which ends the episode. Episodes start on one of the 56 aisle squares chosen uniformly, and $\gamma = 0.9$. An optional *slip* probability replaces the chosen action by a random one.

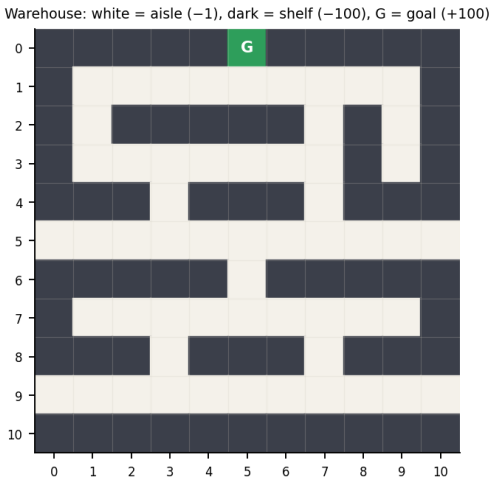


Figure 1: The warehouse: aisles in white, shelves dark, goal (G) in green.

2.2 Ground truth

Value iteration solves the problem in 19 sweeps. As an independent oracle, a breadth-first search gives the true shortest-path length from every square (at most 18 steps). The greedy policy of Q^* follows a shortest path from all 56 starts, so the two oracles agree (Figure 2). Learners are then graded by the *fraction of the 56 starts from which their greedy policy follows a shortest path*, computed exactly rather than by sampling. Where an algorithm has its own target (Q^* for Q-learning, Q^*_ε for SARSA), the error of $V(s) = \max_a Q(s,a)$ to that target is also tracked.

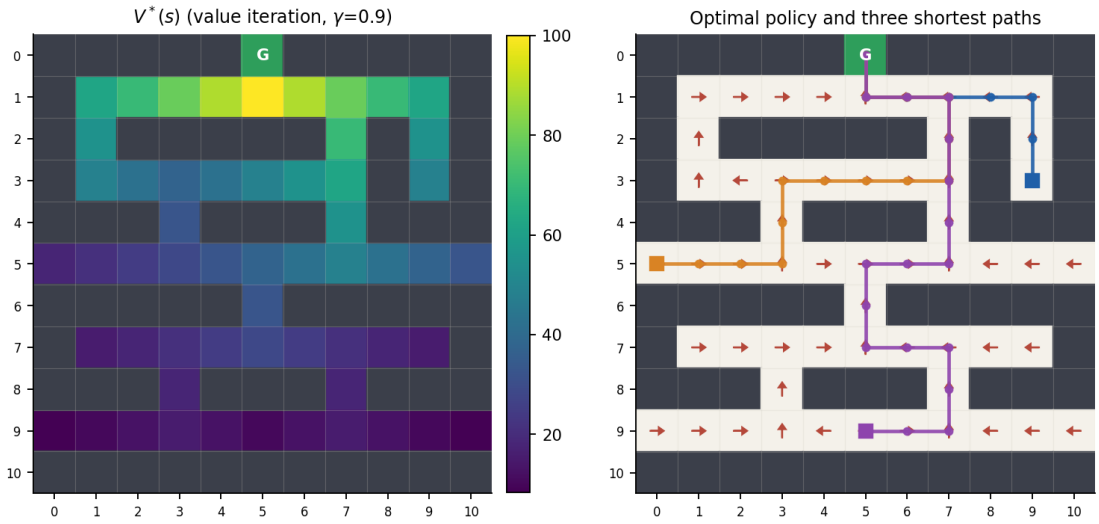


Figure 2: Left: V^* from value iteration. Right: optimal policy and three shortest paths.

2.3 Q-learning versus SARSA

Each configuration is trained for 2000 episodes with $\varepsilon = 0.1$ on 20 independent seeds.

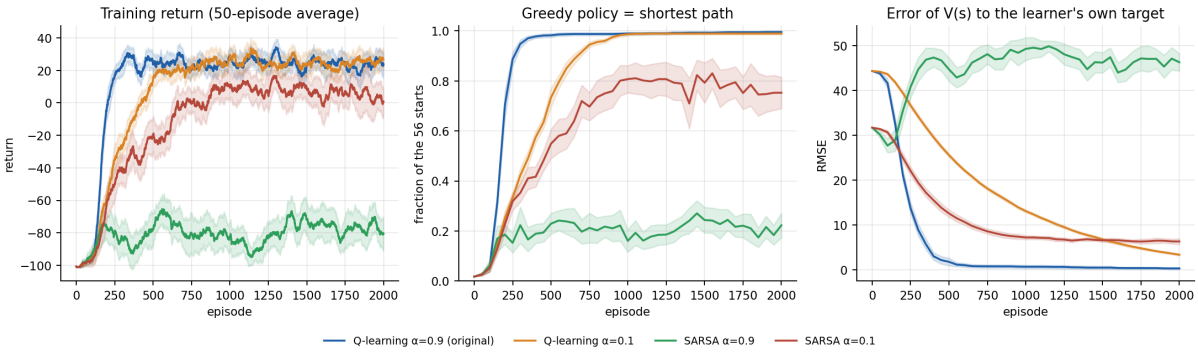


Figure 3: Mean over 20 seeds with 95% confidence bands. Left: training return. Centre: fraction of starts whose greedy path is a shortest path. Right: error of V to the learner's own target.

Configuration	Reaches goal	Shortest path	Episodes to 90%
Q-learning, $\alpha = 0.9$	$100 \pm 0\%$	$99.6 \pm 0.3\%$	300
Q-learning, $\alpha = 0.1$	$100 \pm 0\%$	$98.9 \pm 0.5\%$	700
SARSA, $\alpha = 0.1$	$82.1 \pm 5.3\%$	$75.3 \pm 6.3\%$	never
SARSA, $\alpha = 0.9$	$24.2 \pm 5.9\%$	$22.3 \pm 4.9\%$	never

Table 1: Final greedy policy (mean $\pm$ 95% CI over 20 seeds).

Two things stand out. Q-learning finds shortest paths quickly whatever α is. And its *policy converges long before its value table does*: the greedy policy is already near-optimal while the value error is still large, because only the ordering of the four actions matters.

2.4 Why SARSA struggles: exploring is dangerous

This is not a coding error, and the exact solution shows why. Almost every aisle is lined with shelves, so a random step is often fatal. The exact exploring-agent value V^*_ε is **negative on 28 of the 56 squares** (minimum -24) while V^* is positive everywhere: an agent that keeps exploring is expected to crash before reaching the goal (Figure 4).

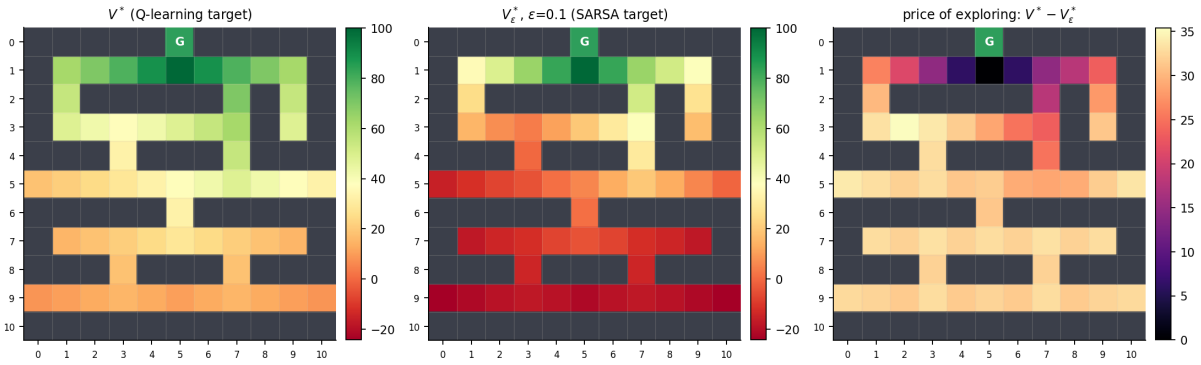


Figure 4: Targets of Q-learning (V^*) and SARSA (V^*_ε, $\varepsilon = 0.1$), and the "price of exploring" $V^* - V^*_\varepsilon$.

Yet the greedy policy of Q^*_ε is a shortest path from all 56 starts: SARSA's *target* is fine, its *estimate* is noisy. Action values differ by only a few units, and a single -100 sample moves an estimate by 100α. Q-learning's max target ignores what exploration would do next and is far less noisy. SARSA's failures are loops (18% of start/seed pairs at $\alpha = 0.1$), never crashes. This is the familiar bias–variance trade-off between on- and off-policy targets.

2.5 Sensitivity to α, ε and γ

Figure 5 reports the fraction of shortest paths after 1000 episodes (8 seeds per cell). Q-learning is robust almost everywhere; it suffers only when ε is large *and* α small (42% at $\alpha = 0.05$, $\varepsilon = 0.4$). SARSA is good only in the low-α, low-ε corner (96–98% at $\alpha \leq 0.1$, $\varepsilon = 0.02$, but 13% at $\alpha = 0.9$, $\varepsilon = 0.1$). The discount barely matters for Q-learning (at least 96% even at $\gamma = 0.5$): with deterministic dynamics and a -1 step cost, even tiny discounted values order the actions correctly. SARSA is lower throughout and not monotonic in γ; with 8 seeds I do not interpret that shape.

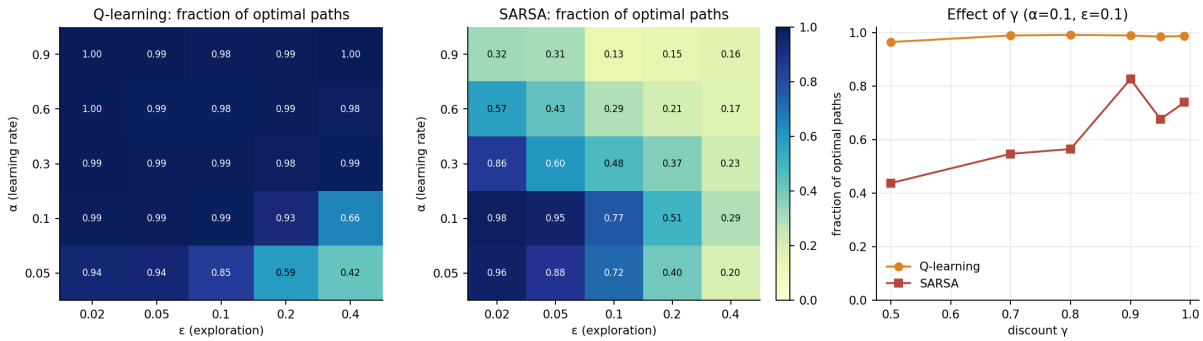


Figure 5: Fraction of shortest paths as a function of α, ε (heat-maps) and γ (right).

2.6 A stochastic warehouse

With probability *slip* the executed action is random. The exact optimum now comes from value iteration with the full transition model, and policies are graded by Monte-Carlo rollouts in the slippery environment (10 seeds $\times$ 500 rollouts).

Goal-reaching rate	slip 0	0.05	0.1	0.2
Exact optimum (dynamic programming)	1.00	0.85	0.68	0.41
Q-learning trained with noise ($\alpha = 0.1$)	1.00	0.77	0.50	0.32
SARSA trained with noise ($\alpha = 0.1$)	0.74	0.68	0.53	0.29
Q-learning trained *without* noise, deployed with it	1.00	0.82	0.67	0.43

Table 2: Even the exact optimum cannot always reach the goal: a random step next to a shelf is fatal.

First, the optimal policy for slip = 0.1 differs from the deterministic shortest-path policy in a **single square out of 56**: in corridors lined with shelves, risk accumulates per step, so the shortest path is also the safest. That is why a policy trained without noise transfers almost perfectly. Second, Q-learning trained *inside* the noisy environment does worse with $\alpha = 0.1$ (50%), and more data does not help (49% after 20 000 episodes): with a constant step size the noise floor is set by α, not by the amount of data. With $\alpha = 0.02$ it recovers the exact optimum (66% against 68%). A decaying step size (Robbins–Monro) would give both fast learning and a vanishing noise floor.

2.7 Policy gradient: REINFORCE

A tabular softmax policy with a per-state baseline (running mean of observed returns) was trained for 10 000 episodes with $\alpha = 0.002$ on 10 seeds.

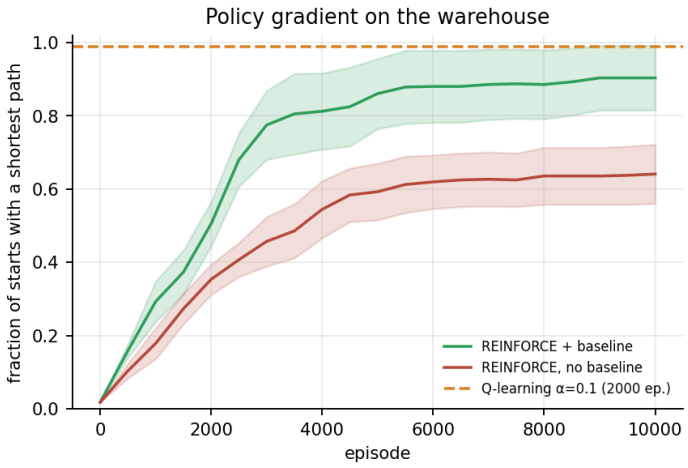


Figure 6: REINFORCE with and without a baseline; dashed: Q-learning ($\alpha = 0.1$, 2000 episodes).

With the baseline, REINFORCE follows a shortest path from $90.4 \pm 8.8\%$ of starts (goal reached from $92.5 \pm 8.4\%$), against $64.1 \pm 8.1\%$ without it: the variance-reduction effect is clear. It still needs about five times more episodes than Q-learning, and results vary more across seeds.

2.8 Limitations

The environment is tiny and tabular, so nothing here addresses function approximation, and the Q-learning/SARSA gap may differ elsewhere. Hyper-parameters in the main comparisons come from the sensitivity study and a small pilot, not from per-algorithm tuning on held-out seeds. Confidence intervals are normal approximations over 8–20 seeds. Finally, ε is constant; a decaying schedule (GLIE) is the standard route to making SARSA converge to the true optimum, and was not explored. Natural next steps are ε and α decay, n-step methods, and a DQN on the same warehouse.

2.9 Reproducibility

The code (`rl_warehouse.py`), the notebook with all outputs, the figures and `results.json` are in the project repository. All randomness is seeded; a full run takes about six minutes on a laptop CPU and reproduces every number in this note.

References

[1] R. S. Sutton, A. G. Barto. *Reinforcement Learning: An Introduction*, 2nd ed., MIT Press, 2018.

[2] C. J. C. H. Watkins, P. Dayan. Q-learning. *Machine Learning* 8, 1992.

[3] R. J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning* 8, 1992.

[4] S. Singh, T. Jaakkola, M. L. Littman, C. Szepesvári. Convergence results for single-step on-policy reinforcement-learning algorithms. *Machine Learning* 38, 2000.